

검증 결과

이 문서는 제출물이 리더보드 제출 결과를 재현하는지 확인한 기록입니다. 1절은 제출 Docker 이미지로 RTX 4090에서 실행한 기본 검증(`scripts/validate.sh`)입니다. 2절은 학습 서버에서 수행한 학습 재현입니다. 3절은 제출물 폴더 구조 그대로 데이터 준비, 학습, 추론 전 과정을 실행한 결과입니다.

1. 이 제출물의 Docker 이미지로 RTX 4090에서 실행 (`scripts/validate.sh`)

- 환경: NVIDIA GeForce RTX 4090, 컨테이너 torch 2.7.1+cu128
- 결과 원문: `validation/result/validation.json`

항목	결과
FLOPs (FlopCounterMode, 1회 forward 전체)	743,777,233,600 (제출 <code>__flops__</code> 와 같습니다)
1회 forward 시간 (BF16, 전처리 제외, warmup 20회 후 100회)	중앙값 30.8 ms , p95 31.3 ms (시간 가중치 ×1.0)
학습 clip 8개, 학습 서버(B200) 예측과 비교	좌표 최대 차이 중앙값 2.2 mm, 최대 4.2 mm, 선택 후보 8/8 일치
테스트 clip 20개, 실제 제출 파일 과 비교	좌표 최대 차이 중앙값 1.5 mm, 선택 후보 18/20 일치, 가중 L2 평균 1.4 mm

선택 후보가 달라진 2개 clip의 가중 차이는 0.3 mm와 14.7 mm입니다. 5초 끝점과 목표점의 거리가 거의 같은 후보가 2개 있을 때, GPU 종류별 BF16 연산 차이로 선택이 바뀌는 경우입니다.

2. 학습 재현 (학습 서버 B200, 같은 코드와 seed)

재현 항목	원래	재현
3단계만 재학습, held-out 모델 (검증 scenario 점수)	0.130237	0.130324
3단계만 재학습, 전체 데이터 모델 (학습 데이터 점수)	0.087320	0.087342
2단계 + 3단계 연쇄 재학습 (학습 데이터 점수, 2단계 / 3단계)	0.091885 / 0.087320	0.091843 / 0.087321
연쇄 재학습 모델의 테스트 예측, 제출본과 비교	—	평균 2.3 cm

- 학습 시작 시점의 값은 비트 단위로 일치합니다.
- 학습 중간에는 곡선이 벌어졌다가 종료 시점에 수렴합니다. 원인은 BF16과 비결정적 CUDA 누적 연산입니다.
- 2단계 사전 점검(smoke) 결과는 다음과 같습니다.
 - 새 프로세스에서 다시 로드한 예측 차이 0.0
 - BN 통계 불변
 - ego 상태를 바꿔도 motion·상태 예측 변화 0.0

3. 제출물 구조 그대로 실행 (학습 영상 준비, 학습 forward/backward, 전체 추론)

제출물의 `tools/assemble.py`로 구성한 경로에서 실행한 결과입니다. 학습 서버(B200)에는 Docker가 없습니다. 그래서 B200에서는 사용자 권한 격리(proot)로 컨테이너와 같은 경로를 구성했고, B200의 PyTorch 2.10 환경을 사용했습니다. 4090에서는 제출 Docker 이미지를 사용했습니다.

항목	환경	결과
학습 영상 준비 (prepare_train.sh, 원본 tar 2개)	4090, Docker	영상 3,600장과 parquet 16개가 학습에 사용한 캐시와 바이트 단위로 모두 일치합니다
학습 영상 준비, 원본 tar 376 개 전체 (tools/build_train_cache.py)	B200	영상 676,800장과 parquet 3,008개가 학습에 사용한 캐시와 바이트 단위로 모두 일치합니다. crop과 카메라 행렬(cache_meta.json)도 같습니다
1단계 smoke (2 update + tune 1,998 평가)	B200	step 1 grad norm 8.681422 (원 기록 8.681418), 평가 d3 12.818012 (원 기록 12.818013)
1단계 smoke, microbatch 2	4090 Docker	학습 샘플 순서 해시가 원 기록과 같습니다. loss 항목은 소수 넷째 자리까지 같고, grad norm은 8.681856(원 8.681418), 평가 d3는 12.818023(원 12.818013)입니다
2단계 smoke (5 update + 평가·검증 probe)	B200	평가 d3 0.106446 (원 기록 0.106435), 종료 코드 0
2단계 smoke + verify, microbatch 2	4090 Docker	5 update의 샘플 순서 해시가 같고, 평가 d3는 0.106446(원 0.106435)입니다. verify에서 strict 재로드 후 예측 차이는 0.0입니다
3단계 smoke (20 update)	B200 / 4090 Docker	두 환경 모두 정상 종료 (64 샘플 평가 0.1633 / 0.1647)
1단계 전체 학습 (24,931 update, 공개 초기값에서 시작)	B200	tune 1,998 평가 0.106857 (원 학습 0.106606, +0.24%). 중간 평가는 벌어졌다가(step 6,345: 0.276 / 0.230) 끝에서 수렴합니다. occupancy IoU 0.3974 (원 0.3980), lane IoU 0.3638 (원 0.3644)입니다. 학습 시간은 3.7시간입니다
3단계 전체 학습 (6,344 update)	B200	tune 1,998 평가 0.087666 (원 학습 0.087320, +0.4%). 동결 몸통 파라미터 427개는 제출 checkpoint와 비트 단위로 같습니다. 학습 대상 planner의 상대 차이는 1.2%입니다. 같은 GPU에서 전체 추론을 동시에 실행했습니다
테스트 1,125 clip 전체 추론 (scripts/infer.sh)	4090, Docker	약 7분이 걸립니다. 선택 후보는 1,118/1,125(99.4%)가 제출본과 같습니다. 같은 후보를 고른 clip의 좌표 최대 차이는 중앙값 1.5 mm, 최대 1.3 cm이고, 전체 가중 차이 평균은 0.9 mm입니다. FLOPs도 같습니다
테스트 1,125 clip 전체 추론 (infer.sh와 같은 명령)	B200	1,125 clip 모두 실제 제출 좌표와 완전히 일치합니다(최대 차이 0). __flops__ 743,777,233,600으로 같습니다
저장한 이미지 파일 (docker save → 기존 이미지 삭제 → docker load)	4090	같은 이미지 ID(sha256:d9c27d55...)로 복원됩니다. validate.sh 결과도 같습니다 (8/8, 18/20, 31.2 ms)

- 1·2단계 코드는 시작할 때 GPU 여유 메모리가 약 174 GiB인지 검사합니다. B200 1단계 smoke는 다른 작업이 28 GB를 쓰는 GPU에서 실행했으므로, 이 사전 검사에 보고되는 여유 메모리만 늘렸습니다. 1단계 전체 학습은 빈 GPU에서 실행해 사전 검사를 그대로 통과했습니다. 1단계 코드는 --gpu 0~3만 받으므로, 이 실행은 CUDA를 해당 GPU에 먼저 연결한 뒤 장치 0으로 보이게 했습니다(Docker의 --gpus와 같은 효과). 두 경우 모두 소스 코드는 수정하지 않았습니다.
- 4090(24 GB)에서 1·2단계를 확인할 때는 tools/small_gpu/sitecustomize.py를 사용했습니다(-e RELCHECK_MICROBATCH=2 -e PYTHONPATH=/shim). microbatch는 8에서 2로 줄이고, 약 174 GiB 여유 메모리 사전 검사는 끝니다. 학습기는 전체 batch(16) 기준 loss normalizer로 microbatch 기울기를 누적하므로, 논리 batch와 계산식은 바뀌지 않습니다. train.sh는 이 도구를 사용하지 않습니다.
- 테스트 전체 추론을 하다가 scripts/infer.sh의 버그를 찾아 고쳤습니다. clip이 많으면 첫 clip을 고르는 ls | head가 pipefail에 걸려 스크립트가 바로 종료됐습니다. 20 clip 검증에서는 드러나지 않았습니다.